

3DY4 Project - Custom Settings for Group 05

For audio processing (both mono and stereo), the four modes of operation are defined in terms of the following parameters. Raw in-phase and quadrature (I/Q) samples are acquired from the radio-frequency (RF) dongle and processed by the software-defined radio (SDR) system at the front-end (input) sample rate, denoted as RF Fs. The frequency-modulated (FM) demodulator operates at an intermediate-frequency sample rate, denoted as IF Fs, which is used at both its input and output; the mono, stereo, and radio data system (RDS) processing paths all receive data from the FM demodulator at IF Fs. Audio samples are produced at the audio (output) sample rate, denoted as Audio Fs, and delivered from the SDR software to an audio player. Finally, the block duration, expressed in milliseconds, determines the number of samples processed per block in each mode of operation. For the project specification corresponding to group 5, the following values apply to RF Fs, IF Fs, Audio Fs, and block duration across the four modes of operation.

Settings for group 05	Mode 0	Mode 1	Mode 2	Mode 3
RF Fs (KSamples/sec)	2400	2304	2400	1152
IF Fs (KSamples/sec)	240	288	240	384
Audio Fs (KSamples/sec)	48	32	44.1	44.1
Block duration (ms)	40	28	60	20

Mode 0 uses the same sample rate values as those used in lab 3 for audio processing. Your group 5 has a unique combination of settings for modes 1, 2, and 3. Therefore, the values described for mode 1 in the project document need to be replaced with those in the above table for modes 1, 2, and 3.

In modes 0 and 2, the IF Fs equals 240 KSamples/sec at the input of the RDS path. Since the RDS path does not use the audio Fs, modes 0 and 2 will be differentiated by changing the number of samples per symbol (SPS) used for clock and data recovery. The project specification for your group 5 will have to support the following two custom values for SPS.

Settings for group 5	Mode 0	Mode 2
Samples per symbol (SPS)	16	31

Note that, depending on the mode of operation, the RDS symbol rate (in samples/sec) at the output of the rational resampler from the RDS demodulator (see Figure 12 from the project document) will be the SPS value from the above table multiplied by 2,375 (symbols/sec).

By default, a packet of data transferred from one module in the signal flow graph to another should carry as many samples as are buffered over the block duration (in ms) listed in the first table. For any filter, the number of partial products accumulated per output sample should be between 75 and 101. This quantity will be referred to as the number of **filter taps**. The expected number of filter taps for each microprocessor (CPU) frequency setting are provided later in this document. This corresponds to the number of taps for all filters, except for resamplers, where the effective number is scaled by the expansion factor. Nonetheless, the number of non-zero partial products for the resampler should still equal the number of filter taps.

Interfacing your program with `rtl_sdr` and `aplay` through Unix pipes

When going live, to enable Unix pipes that stream data into your program from `rtl_sdr` and stream the output of your program to an audio player, such as `aplay`, you should run it as follows from the terminal.

Your program takes two required command line arguments, mode (0, 1, 2, or 3) and processing path (*m* for mono, *s* for stereo, and *r* for RDS). An optional third command line argument may be provided to set the number of **filter taps**. If the third argument is omitted, the number of filter taps defaults to 101, as illustrated in the example below, where the mono path is run in mode 0.

```
rtl_sdr -f 107.1M -s 2400000 - | ./project 0 m | aplay -f S16_LE -c 1 -r 48000
```

It is critical to note that for Unix piping to work as shown above, it is assumed that your program prints any information for the user using `std::cerr`, and **only** the audio samples are written to `stdout`. For the command line arguments for `rtl_sdr` and `aplay`, type the corresponding command in a Linux terminal.

To work in mode 3 with only mono audio enabled and, for example, 83 filter taps, run:

```
rtl_sdr -f 107.1M -s 1152000 - | ./project 3 m 83 | aplay -f S16_LE -c 1 -r 44100
```

After the stereo path has been implemented, you can run it in mode 3 with, for example, 75 filter taps:

```
rtl_sdr -f 107.1M -s 1152000 - | ./project 3 s 75 | aplay -f S16_LE -c 2 -r 44100
```

Note that in the above usage, you will stream twice as much data from your program, and therefore, `aplay` must be configured for two channels.

After implementing the RDS path, you can run it in mode 2 with, for example, 93 filter taps:

```
rtl_sdr -f 107.1M -s 2400000 - | ./project 2 r 93 | aplay -f S16_LE -c 2 -r 44100
```

When the RDS path is enabled, all audio **must** be processed in stereo mode, hence your program must stream its output to `aplay` in two channels. The RDS processing path can be ignored when the software runs in modes 1 and 3. When operating in modes 1 and 3 and the second command line argument is `r`, you may print a message indicating that RDS is not supported.

Emulating streaming during development

While developing your code, you can emulate the streaming of I/Q samples from `rtl_sdr` by redirecting pre-recorded I/Q samples using the `cat` command in Unix. For example, to stream the pre-recorded samples from `iq_samples.raw` to your program running in mode 1 for the mono path (with the default setting of 101 filter taps) do as follows:

```
cat iq_samples.raw | ./project 1 m | aplay -f S16_LE -c 1 -r 32000
```

Note that you must ensure that the I/Q samples have been acquired at the appropriate sample rate, e.g., 2304 KSamples/sec for your project settings for mode 1. To record five seconds, do as follows when the RF dongle is plugged in:

```
rtl_sdr -f 107.1M -s 2304000 -n 11520000 - > iq_samples.raw
```

For modes 0 and 2, the I/Q samples from lab 3 can be reused because they were acquired at 2.4 MSamples/sec and are sufficient to get you started with troubleshooting (including the resampler needed on the mono path in mode 2). If you wish to reuse the lab 3 I/Q samples for other modes at a different sample rate, you can **change** their sample rate using the Python script `fmRateChange.py` in the `model` sub-folder.

Controlling the CPU frequency on Raspberry Pi

To reduce power consumption, you can explicitly control the CPU frequency on the Raspberry Pi 4 devices. Raspberry Pis in the lab are configured to allow student users to scale the CPU frequency. However, **at home only**, after a reboot, you must set the CPU scaling governor to `userspace`:

```
echo userspace | sudo tee /sys/devices/system/cpu/cpu0/cpufreq/scaling_governor
```

Both in the lab and at home, after login, check the current CPU frequency:

```
cat /sys/devices/system/cpu/cpu0/cpufreq/scaling_cur_freq
```

Confirm the pool of available CPU frequencies:

```
cat /sys/devices/system/cpu/cpu0/cpufreq/scaling_available_frequencies
```

Change the CPU frequency to, for example, 1.2 GHz (**at home only**, replace `tee` with `sudo tee`):

```
echo 1200000 | tee /sys/devices/system/cpu/cpu0/cpufreq/scaling_setspeed
```

Confirm the frequency change:

```
cat /sys/devices/system/cpu/cpu0/cpufreq/scaling_cur_freq
```

Note that `scaling_cur_freq` reports the frequency in kHz. If the system refuses to set a requested value, select one of the values listed by `scaling_available_frequencies`.

Monitoring the temperature on Raspberry Pi

A simple temperature measurement using `vcgencmd` can be used as an indirect indicator of relative power dissipation. To monitor the temperature while your program is running on a Raspberry Pi, open a terminal window and check the current temperature (single reading).

```
vcgencmd measure_temp
```

Monitor temperature once per second (recommended to run in a separate terminal):

```
watch -n 1 vcgencmd measure_temp
```

On a Raspberry Pi, power dissipation depends primarily on how many cores are active, and secondarily on the selected operating frequency. Higher power dissipation generates heat, and the measured temperature rises until heat removal to the environment balances the heat being produced. When multiple cores are active, heat is generated over a larger area, increasing the total thermal load. The observed temperature profile also depends on the heat sink contact and size, airflow, and ambient temperature. As a result, temperature typically changes smoothly rather than in sudden steps, but over longer runs (several minutes), higher operating frequencies and greater core activity are expected to produce a higher steady temperature.

Stepwise refinement

To stay organized, a stepwise refinement approach should be followed, starting with a model before proceeding to implementation. Accordingly, you **must** provide **Python** models for mode 0 for both the stereo and RDS paths if your **C++** code is to be assessed. Also, before going live, you must be confident in the correctness of your downsampler and resampler implementations. For this reason, you are required to provide unit tests demonstrating that the slow reference version (derived from first principles of filtering with decimation and expansion implemented as pre- and post-processing functions) is equivalent to the optimized version required for real-time execution on the Raspberry Pi. Based on the same line of reasoning, for incomplete submissions, any **C++** function claimed to be completed must be accompanied by unit tests that prove its correctness and document how it has been evaluated.

When the CPU frequency is scaled down, the signal processing chain must be correspondingly optimized to ensure that the system continues to operate in real time. The primary candidates for optimization are filters operating at high sample rates. To achieve an appropriate balance between computational workload and signal quality, for a given CPU frequency range, your implementation is expected to operate using the following number of filter taps. It is important to note that failure to sustain real-time operation at a given frequency indicates insufficient code optimization.

CPU frequency range	Number of filter taps
$600 \text{ MHz} \leq f_{\text{CPU}} < 900 \text{ MHz}$	75
$900 \text{ MHz} \leq f_{\text{CPU}} < 1.2 \text{ GHz}$	83
$1.2 \text{ GHz} \leq f_{\text{CPU}} < 1.5 \text{ GHz}$	93
$f_{\text{CPU}} \geq 1.5 \text{ GHz}$	101

The number of marks allocated to each of the three processing paths (mono, stereo, and RDS) will be communicated one week prior to the source code submission deadline. Regardless of the final allocation, assessment for each processing path is organized into three components of equal weight. Submitting an individual component does not, by itself, guarantee full credit for that component, as correctness and completeness must be assessed. The three-component structure is intended to reflect the relative priorities and expected development order within each processing path. For the stereo and RDS paths, the first component is the corresponding **Python** model for mode 0, which provides a reference for understanding the signal processing in each case. For the mono path, since the **Python** model for mode 0 was completed during the lab phase of the course, the first component is instead unit tests validating the fast **C++** versions of the downsampler and resampler. For all processing paths, the second component is the full **C++** implementation operating in real time on the Raspberry Pi in mode 0 at a CPU frequency of 1.5 GHz using 101 filter taps. The third component is the **C++** implementation operating in real time on the Raspberry Pi in the remaining modes for each CPU frequency setting, with corresponding filter-tap requirements as defined above.